Not Yet Decided :)

Nirban Das

nirban.das@gmail.com

April 11, 2026

1 Methodology: Time-Sliced Relational Knowledge Graph Fusion for CFRP RUL Prediction

1.1 Problem Formulation

Let the dataset consist of multivariate time-series windows extracted from multiple CFRP specimens. For each sample i, we observe:

$$X_i = [x_{i,1}, x_{i,2}, \ldots, x_{i,T}] \in \mathbb{R}^{T \times d},$$

where T is the sequence length and d is the number of input features. Each sample is associated with:

$$s_i \in \mathcal{S},$$

the specimen identifier, and

$$c_i \in \mathbb{N},$$

the cycle count corresponding to the end of the sequence window.

The primary regression target is the remaining useful life (RUL):

$$y_i \in \mathbb{R}_{\geq 0}.$$

Additionally, a discrete stage label may be available or derived:

$$z_i \in \{0, 1, \ldots, K-1\},$$

where K denotes the number of damage stages.

The task is to learn a predictor of the form

$$f : (X_i, s_i, c_i) \mapsto (\hat{y}_i, \hat{z}_i),$$

where $\hat{y}_i$ is the predicted RUL and $\hat{z}_i$ is the predicted stage label.

The central idea of the proposed method is that the model should not use one static knowledge-graph embedding per specimen. Instead, for each sample, it constructs a *time-sliced ontology subgraph* that contains only knowledge available up to the current cycle c_i, encodes that graph with a relation-aware graph encoder, and fuses the resulting graph memory into the sequence encoder through cross-attention.

1.2 Overview of the Pipeline

The full pipeline can be written as:

$$(X_i, s_i, c_i) \longrightarrow G_i^{\leq c_i} \longrightarrow \text{Relational Graph Encoder} \longrightarrow (M_i, g_i),$$

$$X_i \longrightarrow \text{Transformer Encoder} \longrightarrow Z_i,$$

$$(Z_i, M_i, g_i) \longrightarrow \text{Cross-Attention Fusion} \longrightarrow \widetilde{Z}_i \longrightarrow (\hat{y}_i, \hat{z}_i).$$

Here:

- $G_i^{\leq c_i}$ is the time-sliced ontology subgraph for sample i,
- M_i is the graph memory matrix,
- g_i is the embedding of the target coupon node,
- Z_i is the sequence representation before graph fusion,
- $\widetilde{Z}_i$ is the graph-conditioned sequence representation.

1.3 Time-Sliced Ontology Graph Construction

1.3.1 Full Ontology Graph

Let the populated ontology be represented as a typed knowledge graph:

$$\mathcal{G} = (V, E, \mathcal{R}),$$

where:

- V is the set of nodes,
- $E \subseteq V \times \mathcal{R} \times V$ is the set of directed typed edges,
- $\mathcal{R}$ is the set of relation types.

The implementation parses the ontology and extracts:

1. entity triples,
2. node types,
3. numeric literal attributes,
4. checkpoint cycle counts,
5. coupon-to-checkpoint mappings.

1.3.2 Coupon Resolution

Each specimen identifier s_i is mapped to an ontology coupon node q_i. This mapping is resolved by searching over a small set of coupon naming patterns, such as

$$s_i, \quad L1s_i, \quad L2s_i, \quad L3s_i.$$

1.3.3 Static Neighbourhood

A static structural neighbourhood is first extracted around the coupon node q_i. Let

$$\mathcal{R}_{\text{static}} = \{\texttt{hasLayup}, \texttt{hasMaterial}, \texttt{hasFeature}, \texttt{hasBoundaryCondition}, \texttt{susceptibleTo}, \texttt{correla}$$

Starting from q_i, the algorithm performs graph expansion using only relations in $\mathcal{R}_{\text{static}}$, yielding the static node set

$$V_i^{\text{static}}.$$

This component captures structural prior knowledge that is independent of the current cycle.

1.3.4 Dynamic Neighbourhood

The ontology contains checkpoint nodes linked to coupons, where each checkpoint has a cycle count. For sample i, only checkpoints satisfying

$$\text{cycle}(cp) \leq c_i$$

are considered visible.

Thus the visible checkpoint set is

$$\mathcal{C}_i = \{cp : cp \text{ belongs to } q_i \text{ and } \text{cycle}(cp) \leq c_i\}.$$

From the visible checkpoints, the graph expands only through a restricted dynamic relation set:

$$\mathcal{R}_{\text{dynamic}} = \{\texttt{hasMeasurement}, \texttt{hasDamageObservation}, \texttt{measuredUnder}, \texttt{hasStrainType}\}.$$

This yields the dynamic node set

$$V_i^{\text{dynamic},\leq c_i}.$$

1.3.5 Final Time-Sliced Subgraph

The final node set for sample i is

$$V_i^{\leq c_i} = V_i^{\text{static}} \cup V_i^{\text{dynamic},\leq c_i}.$$

The final edge set is

$$E_i^{\leq c_i} = \{(u, r, v) \in E : u, v \in V_i^{\leq c_i},\ r \notin \mathcal{R}_{\text{excluded}}\},$$

where the excluded relations include target-adjacent or leakage-prone relations such as

$$\mathcal{R}_{\text{excluded}} = \{\texttt{type}, \texttt{influencesRUL}, \texttt{basedOnCoupon}\}.$$

Therefore, the final sample-specific graph is

$$G_i^{\leq c_i} = (V_i^{\leq c_i}, E_i^{\leq c_i}, \mathcal{R}_i).$$

1.3.6 Reverse Edges

For every retained edge

$$(u, r, v),$$

the implementation adds an explicit reverse edge

$$(v, \mathrm{rev}(r), u).$$

This allows message passing in both directions and makes the relational graph encoder more expressive.

1.4 Node Representation

For each node $v \in V_i^{\leq c_i}$, two pieces of information are constructed:

1. a numeric literal feature vector

$$x_v \in \mathbb{R}^{d_g},$$

   where d_g is the total number of discovered numeric ontology attributes;

2. a node type index

$$\tau(v) \in \{0, \ldots, |\mathcal{T}| - 1\}.$$

If a numeric attribute is missing for a node, its corresponding feature value is set to zero. Thus every node is represented by:

$$(x_v, \tau(v)).$$

1.5 Relational Graph Encoder

1.5.1 Initial Node State

Each node is initialised as

$$h_v^{(0)} = W_{\text{feat}} x_v + e_{\tau(v)},$$

where:

- W_{feat} is a learned linear projection,
- $e_{\tau(v)}$ is the learned embedding of the node type.

1.5.2 Relational Graph Convolution

The graph encoder consists of stacked relational graph convolution layers. At layer ℓ, each node v is updated as:

$$h_v^{(\ell+1)} = \sigma\left(W_0^{(\ell)} h_v^{(\ell)} + \sum_{r\in\mathcal{R}} \frac{1}{\deg_r(v)} \sum_{u\in\mathcal{N}_r(v)} W_r^{(\ell)} h_u^{(\ell)}\right),$$

where:

- $\mathcal{N}_r(v)$ is the set of incoming neighbours of v under relation r,
- $W_0^{(\ell)}$ is the self-loop transformation,
- $W_r^{(\ell)}$ is the relation-specific linear map,
- $\deg_r(v)$ is the number of incoming neighbours under relation r,
- $\sigma(\cdot)$ denotes the non-linear transformation used in the implementation.

This encoder is an R-GCN-style relational message passing module.

1.5.3 Graph Outputs

After the final graph layer, the encoder returns:

1. the target coupon embedding

$$g_i = h_{q_i}^{(L)} \in \mathbb{R}^{d_m},$$

   where L is the number of graph layers;

2. a graph memory matrix

$$M_i = \begin{bmatrix} h_{v_1}^{(L)} \\ h_{v_2}^{(L)} \\ \vdots \\ h_{v_{m_i}}^{(L)} \end{bmatrix} \in \mathbb{R}^{m_i \times d_m},$$

   where $v_1, \ldots, v_{m_i}$ are selected memory nodes.

The memory nodes include:

- the coupon node,
- visible checkpoints,
- strain measurement nodes,
- damage observation nodes,
- structural nodes such as layup, feature, boundary condition, and material.

1.6 Sequence Encoder

The time-series branch uses a Transformer encoder.

Given

$$X_i = [x_{i,1}, \ldots, x_{i,T}] \in \mathbb{R}^{T \times d},$$

each timestep is projected into the model dimension:

$$z_{i,t}^{(0)} = W_x x_{i,t}.$$

A sinusoidal positional encoding p_t is added:

$$\widetilde{z}_{i,t}^{(0)} = z_{i,t}^{(0)} + p_t.$$

The sequence is then passed through a Transformer encoder stack:

$$Z_i = [z_{i,1}, z_{i,2}, \ldots, z_{i,T}] \in \mathbb{R}^{T \times d_m}.$$

Thus, Z_i is the latent time-series representation prior to graph fusion.

1.7 Graph-to-Sequence Cross-Attention Fusion

The graph memory is fused into the sequence representation using cross-attention, where:

- queries come from the sequence states Z_i,
- keys and values come from the graph memory M_i.

Formally,

$$Q_i = Z_i W_Q, \qquad K_i = M_i W_K, \qquad V_i = M_i W_V.$$

The attention weights are:

$$A_i = \mathrm{softmax}\left(\frac{Q_i K_i^\top}{\sqrt{d_m}}\right).$$

The graph-conditioned context is:

$$C_i = A_i V_i.$$

Finally, the fused sequence representation is obtained by residual fusion and layer normalisation:

$$\widetilde{Z}_i = \mathrm{LayerNorm}(Z_i + C_i).$$

Since different samples may have different numbers of memory nodes m_i, the graph memories are padded within a batch before the attention computation.

1.8 Joint Representation and Prediction Heads

After cross-attention fusion, the sequence branch is mean-pooled over time:

$$u_i = \frac{1}{T} \sum_{t=1}^{T} \widetilde{z}_{i,t}.$$

The final joint representation is

$$r_i = [u_i \,\|\, g_i] \in \mathbb{R}^{2d_m},$$

where $\|$ denotes vector concatenation.

Two prediction heads are used.

1.8.1 RUL Regression Head

The RUL prediction is:

$$\hat{y}_i = \psi_{\mathrm{rul}}(r_i),$$

where ψ_{rul} is a multilayer perceptron with scalar output.

1.8.2 Stage Classification Head

The stage prediction is:

$$\hat{z}_i = \psi_{\text{stage}}(r_i),$$

where ψ_{stage} is a multilayer perceptron producing logits over the K stage classes.

1.9 Auxiliary KG Regularisation

The implementation includes an optional TransE-style auxiliary loss to regularise graph structure.

For a triple (h, r, t), let:

$$e_h, \quad e_r, \quad e_t$$

denote the source node, relation, and target node embeddings, respectively.

The positive triple score is:

$$s^+(h, r, t) = \|e_h + e_r - e_t\|_2.$$

For a corrupted negative triple (h, r, t^-), the score is:

$$s^-(h, r, t^-) = \|e_h + e_r - e_{t^-}\|_2.$$

The margin-based KG loss is:

$$\mathcal{L}_{\text{kg}} = \max\big(0, \gamma + s^+(h, r, t) - s^-(h, r, t^-)\big).$$

This auxiliary loss encourages the learned graph embeddings to preserve relational structure, but it does not replace the supervised prognostic objective.

1.10 Training Objective

The full training objective is:

$$\mathcal{L} = \mathcal{L}_{\text{rul}} + \lambda_{\text{stage}}\mathcal{L}_{\text{stage}} + \lambda_{\text{kg}}\mathcal{L}_{\text{kg}}.$$

1.10.1 RUL Loss

The RUL regression term is a Smooth L1 loss:

$$\mathcal{L}_{\text{rul}} = \text{SmoothL1}(y_i, \hat{y}_i).$$

1.10.2 Stage Loss

The stage classification term is cross-entropy:

$$\mathcal{L}_{\text{stage}} = -\sum_{k=1}^{K} \mathbf{1}[z_i = k] \log p_{ik},$$

where p_{ik} is the predicted probability of class k.

Therefore, the final implemented objective is:

$$\boxed{\mathcal{L} = \text{SmoothL1}(y_i, \hat{y}_i) + \lambda_{\text{stage}} \, \text{CE}(z_i, \hat{z}_i) + \lambda_{\text{kg}} \mathcal{L}_{\text{kg}}}$$

1.11 Grouped-Split Experimental Protocol

The experimental protocol uses specimen-grouped splitting to prevent information leakage across windows from the same specimen.

Let the set of specimen identifiers be

$$\mathcal{S} = \{s_1, \ldots, s_M\}.$$

An outer grouped split is performed such that

$$\mathcal{S}_{\text{train}} \cap \mathcal{S}_{\text{test}} = \varnothing.$$

Within each outer training fold, a grouped validation split is created, again ensuring that no specimen appears in both training and validation.

Thus, for every fold:

$$\mathcal{S}_{\text{train}} \cap \mathcal{S}_{\text{val}} = \varnothing, \qquad \mathcal{S}_{\text{train}} \cap \mathcal{S}_{\text{test}} = \varnothing, \qquad \mathcal{S}_{\text{val}} \cap \mathcal{S}_{\text{test}} = \varnothing.$$

1.12 Train-Only Scaling

All preprocessing is fitted on the training fold only.

1.12.1 Sequence Standardisation

For each feature dimension j, the training mean and standard deviation are:

$$\mu_j = \frac{1}{N_{\text{train}} T} \sum x_{i,t,j}, \qquad \sigma_j = \sqrt{\frac{1}{N_{\text{train}} T} \sum (x_{i,t,j} - \mu_j)^2}.$$

Then:

$$x'_{i,t,j} = \frac{x_{i,t,j} - \mu_j}{\sigma_j}.$$

The same (μ_j, σ_j) are applied to the validation and test folds.

1.12.2 Target Standardisation

For the RUL target:

$$\mu_y = \frac{1}{N_{\text{train}}} \sum y_i, \qquad \sigma_y = \sqrt{\frac{1}{N_{\text{train}}} \sum (y_i - \mu_y)^2},$$

and:

$$y_i' = \frac{y_i - \mu_y}{\sigma_y}.$$

Predictions are inverse-transformed to the original RUL scale during evaluation.

1.13 Baselines and Evaluation

The current implementation evaluates two models:

1. **Plain Transformer Baseline:**

$$X_i \longrightarrow \text{Transformer} \longrightarrow (\hat{y}_i, \hat{z}_i),$$

   with no graph input.

2. **Time-Sliced Relational Memory Fusion Model:**

$$(X_i, s_i, c_i) \longrightarrow G_i^{\leq c_i} \longrightarrow \text{Relational Graph Encoder} \longrightarrow \text{Cross-Attention Fusion} \longrightarrow (\hat{y}_i, \hat{z}_i).$$

The reported evaluation metrics are:

Mean Absolute Error (MAE)

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^{N} |y_i - \hat{y}_i|.$$

Root Mean Squared Error (RMSE)

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2}.$$

Per-Specimen MAE For each specimen s,

$$\mathrm{MAE}_s = \frac{1}{N_s} \sum_{i:s_i=s} |y_i - \hat{y}_i|.$$

1.14 Summary of the Implemented Method

The implemented method can be summarised compactly as:

$$\boxed{\hat{y}_i, \hat{z}_i = \Psi\Big(\mathrm{Fuse}\big(\mathrm{Transformer}(X_i), \mathrm{RelGraphEnc}(G_i^{\leq c_i})\big)\Big)}$$

with the time-sliced graph defined as

$$\boxed{G_i^{\leq c_i} = \text{ontology subgraph visible for specimen } s_i \text{ up to cycle } c_i}$$

and trained under the grouped-split constraint

$$\boxed{\mathcal{S}_{\mathrm{train}} \cap \mathcal{S}_{\mathrm{test}} = \varnothing.}$$

The contribution of the implemented pipeline is therefore threefold:

1. it replaces static full-coupon graph context with leakage-safe time-sliced graph context;
2. it uses relation-aware graph encoding instead of relation-agnostic neighbourhood embeddings;
3. it fuses graph memory into the sequence branch through cross-attention rather than a constant additive broadcast.